

# Week 04 Face-to-Face Class Outline

## Topic 1: Representational State Transfer (REST)

1. REST captures the characteristics of the Web
  - a. URI/URL addressable resources
  - b. HTTP protocol
  - c. Request-Response-Display cycle
2. Relationship and difference between REST and HTTP
  - a. REST is not a standard, but HTTP is a standard (e.g., HTTP/1.1 is standardized in RFC 2616: <https://datatracker.ietf.org/doc/html/rfc2616>)
  - b. REST uses several standards including HTTP, URL, MIME, etc
  - c. REST uses HTTP methods but may apply different interpretations

|             | Methods                                                 |                                                                                 |                             |                             |
|-------------|---------------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------|-----------------------------|
|             | <i><b>GET</b></i>                                       | <i><b>POST</b></i>                                                              | <i><b>PUT</b></i>           | <i><b>DELETE</b></i>        |
| <b>HTTP</b> | Retrieve a content (parameters are transmitted via URL) | Retrieve a content (parameters are transmitted via entity body of HTTP request) | Upload a content            | Delete a content            |
| <b>REST</b> | Retrieve a resource                                     | Create a resource                                                               | Update an existing resource | Delete an existing resource |

3. REST uses three main concepts
  - a. Nouns (resources addressed by URI/URL)
  - b. Verbs (e.g., HTTP GET)
  - c. Representations (e.g., XML, JSON)
4. Structured URI/URL for organizing/addressing resources. Examples:
  - a. <http://localhost/classes> contains a list of classes offered
  - b. <http://localhost/classes/sc4031> contains metainfo of course SC4031
  - c. <http://localhost/classes/sc4031/students> contains a list of students registered for SC4031

## Topic 2: Constrained Application Protocol (CoAP)

1. CoAP is based on
  - a. the REST principle
  - b. a subset of HTTP functionality re-designed for resource-constrained devices
2. CoAP is an application protocol on top of UDP for efficiency. Reliability (primarily, its acknowledgement mechanism) is implemented at the application layer, rather than transport layer.
3. CoAP employs four message types
  - a. Confirmable: requires ACK
  - b. Non-confirmable: no ACK needed
  - c. ACK
  - d. Reset: used when a confirmable message has been received but context is missing
4. A proxy gateway can interface the TCP/IP-based Internet and the CoAP-based IoT network (related topic: “adaptation” in Week 07 recorded lecture).
5. CoAP is lightest in overhead among all cloud-IoT communication protocols introduced in this course

## Topic 3: Message Queuing Telemetry Transport (MQTT)

1. A MQTT system has
  - a. A broker
  - b. Subscribers categorized by topics
  - c. Publishers categorized by topics
2. MQTT can be viewed as a middleware between the TCP protocol and application-layer protocols
3. Topic is identified by URL
  - a. Example: home/rooms/kitchen/temperature
4. The broker multi-casts data in a topic published by a publisher to the subscribers of the topic
  - a. Example 1: by `clientX.subscribe('home/rooms/kitchen/temperature')`, the `clientX` is registered to the broker as a subscriber of the topic
  - b. Example: by `clientY.publish('home/rooms/kitchen/temperature', 29)`, the `clientY` publishes a data of 29 to the topic. Then, all subscribers of the topic will receive the data
5. Comparison between CoAP and MQTT

|      | Communication pattern | Transport layer | Reliability                                       | Overhead |
|------|-----------------------|-----------------|---------------------------------------------------|----------|
| CoAP | One-to-one            | UDP             | Some reliability implemented at application layer | Low      |
| MQTT | Many-to-many          | TCP             | Reliable                                          | High     |